

Advanced Data Analytics — Lecture 9

Applications in NLP

Simon Scheidegger
Department of Economics, University of Lausanne, Switzerland

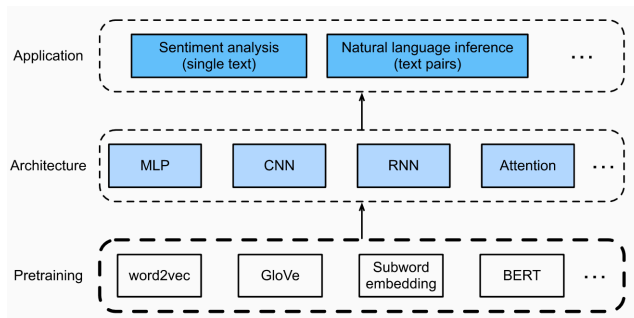
November 10th, 2025 | 10:15 - 12:00: Internef 126 | 16:30 - 18:00: Anthropole 3185

Roadmap

- ▶ Embeddings
- ▶ Language Modeling

Motivation

- In our discussion on deep learning for text, we have mainly focused on the middle part of this picture:



- All neural network models assume real-valued vector inputs, and we have assumed that there is some magical way to convert discrete data (such as text) to a form that neural networks can process.
- Today, we will focus on the bottom part. Where do the word encodings come from? And how do they interact with the rest of the learning?

- ▶ The easiest way to encode words/tokens into real-valued vectors is one that we have already used for image-classification type applications: one-hot encoding.
- ▶ Pros: this is dead simple to understand and implement.
- ▶ Cons: there are two major drawbacks of using one-hot encodings.
- ▶ Each encoding can become very high dimensional. By definition, the encoded vectors are now the size of the vocabulary/dictionary. At the character level this is fine; at the word level it becomes very difficult; and at any higher level the space of symbols becomes combinatorially large.
- ▶ More than just computation: one-hot encodings do not capture semantic similarities (all words are equally far in L1/L2/Hamming distance than every other word). There are ways to augment one-hot vectors with more information, but we will not be considering them in this lecture.
- ▶ It would be nice to have **similar words share similar features** (where the meaning of “similar” depends on the language and/or context).

word2vec (II)

[https:](https://towardsdatascience.com/uncovering-the-pioneering-journey-of-word2vec-and-the-state-of-ai-science-an-in-depth-interview-fbca93d8f4ff)

[//towardsdatascience.com/uncovering-the-pioneering-journey-of-word2vec-and-the-state-of-ai-science-an-in-depth-interview-fbca93d8f4ff](https://towardsdatascience.com/uncovering-the-pioneering-journey-of-word2vec-and-the-state-of-ai-science-an-in-depth-interview-fbca93d8f4ff)

- ▶ This was recognized by early NLP researchers. Before the mid-2000s, a host of encoding methods were proposed, including latent semantic analysis (LSA).
- ▶ The term “embedding” itself was proposed in 2003. It means that we are, quite literally, *embedding* input words in a vector space.
- ▶ It came from a paper that proposed the first ever neural model for creating vector representations of words (Bengio et al., 2003)
- ▶ Later, in 2013, Mikolov et al. published word2vec, inspired by this earlier work. It was a breakthrough in neural NLP and is widely regarded as the “turn of the century” which eventually led us to things like ChatGPT.
- ▶ Word2vec is a word embedding framework that uses one of two approaches: skip-grams and continuous bag-of-words.

Skip-grams

- ▶ In skip-grams, each word has two vector embeddings: v_i and u_i .
- ▶ Let us first motivate why we need two such embeddings. As a running example, we will keep in mind a sentence such as:

“It is raining right now”.

and imagine it being represented as a sequence of words

x_1, x_2, x_3, x_4, x_5

- ▶ We already discussed n – *grams* briefly before while motivating language models. For $n = 2$, these are the joint probabilities

$P(x_1, x_2), P(x_2, x_3), \dots$

each of which can be empirically calculated by counting the number of co-occurrences of pairs of words in a database.

Skip-grams (II)

- ▶ The term “skip-gram” comes from the fact that we consider conditional probabilities that are not-consecutive, i.e., words can be skipped over.
- ▶ The reason for exploring relationships between non-consecutive words goes back to the non-local, long-range dependency structure of natural languages.
- ▶ In this case, the factorization is done with respect to the “center” (or “target”) word of the sequence; the other words are called “context” words.
- ▶ So the above factorization becomes:

$$P(x_1|x_3) \cdot P(x_2|x_3) \cdot P(x_4|x_3) \cdot P(x_5|x_3)$$

- ▶ ntuitively, these probabilities tell us: “if a word x_i appears in a sentence, how likely is it that the word x_j will appear in its vicinity?” Here, “vicinity” would mean a window of some fixed size.

Skip-grams (III)

- ▶ Having defined non-local conditional probabilities, the algorithmic question now becomes: how to estimate them?
- ▶ Again, one could just use the frequency of co-occurrence counts in some large text corpus. However, we will depart from the standard approach, and instead train a simple neural network that predicts

$$P(x_j|x_i).$$

- ▶ The network will be two layers deep (i.e., a single hidden layer of neurons with linear activations), followed by a softmax.

Skip-grams (IV)

- ▶ Some more details about this network. Say we have a dictionary of N words.
- ▶ The input is a one-hot encoding x_i of any given word i (so, N input neurons).
- ▶ The output is a vector of pre-softmax logits (so, N output neurons).
- ▶ We can imagine (say) a hidden layer of d (linear) neurons.
- ▶ So if we call $V \in \mathbb{R}^{d \times N}$ and $U \in \mathbb{R}^{d \times N}$ the two layers, then the conditional probability of any context word given the center is given by:

$$\begin{aligned} P(x_j|x_i) &= \text{softmax}(U^T V x_i) \\ &= \text{softmax}(U^T v_i) \\ &= \text{softmax}([u_1^T v_i; u_2^T v_i; \dots u_N^T v_i]). \end{aligned}$$

Skip-grams (V)

- ▶ So examining the rows of U and V give us precisely what we want — the word embeddings for the target and the context words respectively.
- ▶ Typically, $d \ll N$, so the embedding dimension is much smaller than the size of the vocabulary.
- ▶ Using the rows of U and V as embeddings also intuitively makes sense: similar words/synonyms should give us similar output probabilities, and in order for two outputs to be similar, both target and context probabilities must match.

Skip-grams (VI)

- ▶ How do we train this network?
- ▶ First, we need to define a loss function.
- ▶ We can just use the standard cross-entropy loss, where the network is fed pairs of words (one-hot encoded) as data-label pairs.
- ▶ So for a particular pair of target-context words i and j , we get the loss term:

$$l(i, j) = u_j^T v_i - \log \left(\sum_j \exp(u_j^T v_i) \right)$$

whose derivative can then be used to update all the weights.

Skip-grams (VII)

- ▶ There are a more few issues here to be considered.
- ▶ In English, for example, there are about $N = 10K$ commonly used words.
- ▶ So we already have approximately $6M$ weights to learn.
- ▶ Second, training can be extremely slow, since for every sample pair we have to touch all the weights.
- ▶ The word2vec paper did a few extra hacks (hierarchical softmax, negative sampling) to make this work, which we won't dive into here – more details in an NLP course perhaps.

Continuous Bag of Words (CBOW)

- ▶ The CBOW model is very similar to the skip-gram model, so we won't get into too much detail.
- ▶ The main difference is that the CBOW model flips things around: instead of the center word defining the context, the context words are used to predict the target.
- ▶ So the conditional probabilities become:

$$P(x_3|x_1, x_2, x_4, x_5)$$

which cannot be easily factorized the way we did so above. But the expression remains similar if we approximate the embedding of the context as the (vector) average of the individual embeddings:

$$P(x_i|x_1, \dots, x_j, \dots) = \frac{\exp(u_i^T \text{Ave}(v_j))}{\sum_i \exp(u_i^T \text{Ave}(v_j))}.$$

- ▶ Given this approximation of the conditional probabilities, the training is done just the same way as described above using the cross-entropy loss.
- ▶ Which embedding is better? Both are roughly equivalent, and we could use one or the other.

- ▶ The main problem with the word2vec framework is that both skip-gram and CBOW models rely on predicting output probabilities, and hence have to be trained with the cross-entropy loss.
- ▶ For very large dictionaries, calculating cross-entropy can be troublesome: each gradient update requires computing softmaxes (and hence calculating all the outputs and marginalizing over them). Global vector (GloVe) embeddings resolve this in a slightly different manner.

GloVe (II)

- ▶ The idea is to use matrix factorization (a la PCA), and since it is not neural network-based we won't go into too much detail here: take an NLP class if interested.
- ▶ The main steps are as follows: We construct a word-context co-occurrence matrix and try to factorize it using PCA (i.e., find the low-rank decomposition that minimizes the reconstruction loss).
- ▶ Not trivial, but this is a very sparse matrix! Can train using SGD-type methods.
- ▶ Word distributions have a long tail, so very common words will dictate the loss function. Log probabilities are used to make things more equitable.

GloVe (III)

- In practice, a modified weighted form of the reconstruction loss is used:

$$L(U, V, b, c) = \sum_{i,j} f(x_{ij})(u_i^t v_j + b_i + c_j - \log x_{ij})^2$$

where $f(x_{ij}) = 1$ for reasonable x_{ij} but quickly goes to 0 if x_{ij} gets close to zero.

- This avoids the possibility that large (negative) values in the log probabilities significantly influence the loss function.

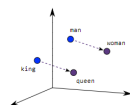
Cosine Similarity

- ▶ We have said previously that the goal of this complicated machinery is to preserve semantic similarity in our word representations: i.e. have words that are more similar in meaning be closer together in representation.
- ▶ Both of the discussed methods achieve this goal spectacularly.
- ▶ The **cosine** of the angle between two of the vectors that they produce can be interpreted as the level of similarity between the two words.

$$\cos(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

Arithmetic Analogies

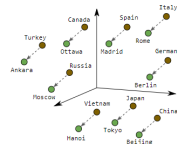
- Furthermore, they also preserve **logical analogies**.
- How? With **vector arithmetic**.
- For example, $\text{vector}(\text{'Paris'}) - \text{vector}(\text{'France'}) + \text{vector}(\text{'Italy'})$ results in a vector that is very close to $\text{vector}(\text{'Rome'})$,
- and $\text{vector}(\text{'king'}) - \text{vector}(\text{'man'}) + \text{vector}(\text{'woman'})$ is close to $\text{vector}(\text{'queen'})$.
- These analogies are also preserved when projected in 2D or 3D space.



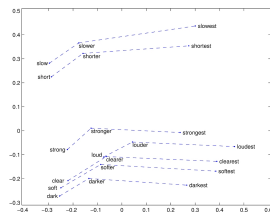
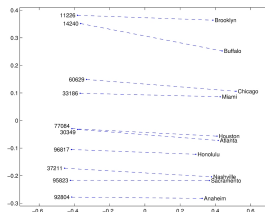
Male-Female



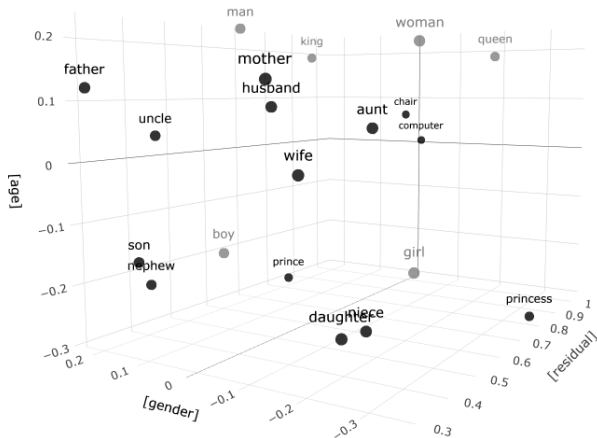
Verb Tense



Country-Capital



Projection of embeddings



<https://www.cs.cmu.edu/~dst/WordEmbeddingDemo/index.html>

The axes are interpreted as semantic directions.

The result

- ▶ In practice, these methods can be used to produce dense vectors that will represent words of a given vocabulary.
- ▶ For example, GloVe authors encoded a 400k collection of 300-dimensional vectors for English words by training the model on the entire text of the English Wikipedia.
- ▶ This is a remarkable level of dimensionality reduction compared to one-hot encoding.
- ▶ You can download pre-trained word2vec or GloVe as a file, replace the words in your dataset with the vectors, and use them as input to other models for more specific tasks.
- ▶ Or you can re-train them on your data.

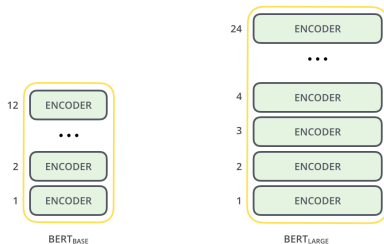
- ▶ The main problem with GloVe/word2vec is that the word embeddings are context-independent. Recall that if we want to get a skip-gram embedding of a word, we one-hot encode it and look at the corresponding input- and output-layer weights in the above two-layer architecture.
- ▶ However, words (particularly in languages such as English) are context-dependent. For example, consider a sentence such as “Fish fish fish” — there are three identical words here, but the context shows that each has rather different meanings. Can we somehow get embeddings that look not just at word-level semantics but also at their usage in a given sentence?

ELMo (II)

- ▶ ELMo (Embeddings from Language Models) does this. Just as how we motivated RNNs/LSTMs as possible architectures that can capture context in a sequence of inputs, similarly we can replace the simple feedforward architecture of word2vec with recurrent architectures.
- ▶ Specifically, ELMo proposes to produce word embeddings by looking at the entire sentence, both left-to-right and right-to-left. It achieves this via bi-directional LSTMs: it looks at the hidden layer representations (states) for both the left-to-right and right-to-left LSTMs and takes a weighted linear combination of them as the word embedding for each word in the input. The weights are left as trainable parameters used by downstream tasks (such as classification or sentence prediction) for further fine-tuning.
- ▶ The choice of the loss function is important; ELMo uses next-word-prediction (NWP) as the task of choice using the cross-entropy loss.

BERT

- ▶ The next natural progression was to replace the bidirectional LSTM encoding used by ELMo with Transformers.
- ▶ This led to BERT (bidirectional encoder representations from transformers).
- ▶ Previously, we saw the full Transformer architecture, which consisted of two main blocks: encoder and decoder. This architecture functioned as a full sequence-to-sequence model (for example, for machine translation).
- ▶ BERT utilizes only the first half of it, thus producing *encoded* representations of input text.
- ▶ The main ingredients (over and above those described above) include:
 - ▶ Replacing LSTMs with transformer blocks. The output of each encoder layer in each token's path can be viewed as a feature embedding of that token.



BERT (II): Pre-training

- ▶ To summarize, the input of a **trained** BERT model is a *tokenized* string of words. BERT uses word-piece tokenization, which is somewhere in between character-level and word-level encoding. This is useful for languages like English. For example, the word “walking” is broken into two pieces: “walk” and “ing”, each of which are considered separate.
- ▶ Its output is **an information-rich dense encoding** of said string. The BERT model produces embeddings of 768 dimensions, and it can process texts up to 512 tokens long.
- ▶ These embeddings have proven to be a highly useful representation for various tasks of processing and understanding human language (more on that later).
- ▶ However, to get the model to this point, the authors have come up with a special ***pre-training task*** of ***masked language modeling***.
- ▶ This training objective allows the model to learn **highly generalizable representations of language** (can you think about why this is true?)

Language Modeling?

- ▶ So far, we have only mentioned tasks that language models are trained on.
- ▶ But formally, the term “**language modeling**” refers to the task of placing a probability distribution over all tokens of a specified vocabulary, given a sequence of input tokens.
- ▶ In practice, first “language models” were trained to predict the next word in a sentence, given its “left-hand side” — the next-word-prediction task of ELMo.
- ▶ This task was used successfully for the GPT family of models. However, in this context, it has a conceptual limitation.
- ▶ Such models are incapable of considering all context that exists in a text.
- ▶ This can be detrimental to the overall quality of representation (remember that we are currently talking about representing text in a semantically rich way!)
- ▶ For example, we receive an input of I **went to the** and want to predict the next word. However, with no context, this is basically impossible, as there are way too many versions.
- ▶ If we knew that the missing word was followed by “for a walk”, we would have had a much better chance.

Masked Language Modeling

- ▶ This is why Masked Language Modeling is a task in which the modeling objective gets “turned around”:
- ▶ Now, the model randomly replaces a small set percentage of input tokens with `<MASK>`. Then, it is trained to predict the probabilities of taking the place of `<MASK>` for all tokens of the vocabulary.
- ▶ Hence, the “bidirectional” part of the name, as attention for the masked tokens is now calculated with both “LHS” and “RHS” of the text in mind (recall last lecture!)

I went to the `<MASK>` for a walk.



park: **0.998**

...

doctor: 0.459

...

cat: **0.005**

Next Sentence Prediction

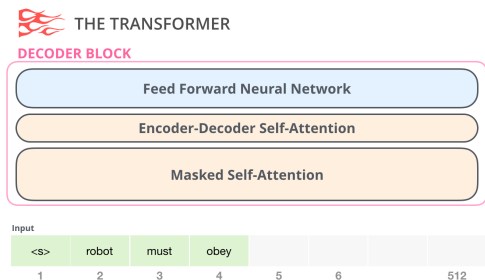
- ▶ The masked modeling task is very useful when we want to understand the meaning that words carry, which is useful for tasks like sentiment analysis.
- ▶ However, some important NLP tasks, such as Question Answering, need the model to be able to deal with logical inference, and that is not directly captured by the masked modeling task.
- ▶ So, another pre-training task for BERT is Next Sentence Prediction:
- ▶ The model is fed pairs of concatenated sentences, and trained to output whether sentence 2 follows sentence 1 in a text or not.
- ▶ This type of simple pre-training task has allowed BERT once again to beat state-of-the-art question answering models back in 2018.

Pre-training → Fine-tuning

- ▶ Training the model in such **unsupervised** fashion allows it to become very good at creating language representations without annotated data.
- ▶ To use them in any meaningful NLP task, we can go two ways:
- ▶ 1. encode text with the model, obtain a list of vectors and train a separate neural network on them and other, additional features. This is called a **feature extraction** approach, and it has its benefits:
 - ▶ Conceptually, some tasks require domain-specific features
 - ▶ Computationally, encoding data once and running a set of small models on it is much cheaper overall
- ▶ 2. swapping the output layer of the model to output required labels, and training the *already trained* model further on task-specific data to adapt its weights to the task. This is called ***fine-tuning***.
- ▶ Both of these ideas are referred to as *transfer learning*: using knowledge learned for one task for another task.

Generative P retrained T ransformer

- ▶ Recall: BERT is the *encoder* part of the original Transformer.
- ▶ GPT, however, is the *decoder*: the second half of the original architecture.
- ▶ Its development was only initially focused on the same tasks as BERT, however, further on, the version provided by OpenAI became the best text generator there is.
- ▶ How does it work?



<https://jalammar.github.io/illustrated-gpt2/> — a very interesting and detailed explanation of how GPT works

GPT-1

<https://openai.com/research/language-unsupervised>

- ▶ The GPT-1 model is trained to predict the *next word in the input sequence*, unlike BERT.
- ▶ Thus, it can only incorporate the information found on the "left-hand side" of the sentence.
- ▶ This makes it more fit for plainly generating text, and not predicting probabilities for a downstream task such as we have discussed.
- ▶ In the original BERT paper by Google, BERT beats GPT-1 based directly on this point: it is harder to fine-tune GPT than BERT.
- ▶ However, many NLP tasks can be reformulated as text generation:
- ▶ For example, a translation training example can be written as the sequence (translate to french, english text, french text).

GPT-2

<https://openai.com/research/better-language-models/>

- ▶ It turns out that if you pre-train a generative model on large enough general language data (such as the entire Internet, or even less — just 40 GB of text for GPT-2!), the model will be able to perform tasks ***without ever being fine-tuned for them.***
- ▶ For example, there are translations from English to French naturally occurring in text found on the internet, such as:
- ▶ `'I'm not the cleverest man in the world, but like they say in French: Je ne suis pas un imbecile [I'm not a fool].`
- ▶ Another example is question answering, where in the text, the answer directly follows the question: Who wrote the book the origin of species? Charles Darwin
- ▶ The GPT-2 model **did not have to be additionally trained at all** to tackle these tasks and many more.
- ▶ This phenomenon is referred to as *zero-shot* task transfer, or, later on, *meta-learning*.
- ▶ It was, essentially, the first real success of *unsupervised learning* in the field of NLP, which was a breakthrough: before this, expensive and hard to obtain annotated data was usually required.

GPT-3

<https://arxiv.org/abs/2005.14165>

- ▶ With the advancements in hardware over the following years (GPT-2 came out in early 2019), GPT-3 was already trained on 600GB of web text and additional book data in mid-2020.
- ▶ GPT-3 was also able to do task-specific text completion in a “*few-shot*” fashion: that is, it could be shown some examples of tasks (without weight updates, at inference time!) and from them infer how to do the task by itself.

The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← example
3 cheese => ..... ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => ..... ← prompt
```

Traditional fine-tuning (not used for GPT-3)

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



Alignment: InstructGPT

<https://openai.com/research/instruction-following>

- ▶ GPT-3 was already able to work in a fashion that is currently very familiar to us: you **prompt** the model to do something, and it **completes** your request.
- ▶ However, the results often left much to be desired.
- ▶ In order to make the generated text more compliant with the demands of the users, researchers at OpenAI came up with **reinforcement learning from human feedback**.
- ▶ It was used to **align** the model's output with human expectations.
- ▶ InstructGPT was the "intermediate" model between GPT-3 and ChatGPT. While much smaller, it generated outputs that were more agreeable for human users.

Prompt	<i>Explain the moon landing to a 6 year old in a few sentences.</i>
Completion	GPT-3 Explain the theory of gravity to a 6 year old. Explain the theory of relativity to a 6 year old in a few sentences. Explain the big bang theory to a 6 year old. Explain evolution to a 6 year old. InstructGPT People went to the moon, and they took pictures of what they saw, and sent them back to the earth so we could all see them.

RLHF

<https://arxiv.org/abs/2203.02155>

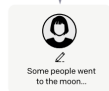
Step 1

**Collect demonstration data,
and train a supervised policy.**

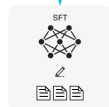
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



This data is used
to fine-tune GPT-3
with supervised
learning.



Step 2

**Collect comparison data,
and train a reward model.**

A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

A new prompt
is sampled from
the dataset.



The policy
generates
an output.

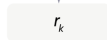


Once upon a time...

The reward model
calculates a
reward for
the output.



The reward is
used to update
the policy
using PPO.



RLHF as proposed by OpenAI

GPT-4

<https://openai.com/research/gpt-4>

- ▶ This line of work has culminated in ChatGPT: GPT-3.5 and finally, GPT-4.
- ▶ GPT-3.5 is the “test run” of the RLHF training procedure (in the words of researchers themselves). Unlike Instruct, it is also optimized to have an actual friendly conversation with the user.
- ▶ GPT-4 is the most advanced model so far, with the most impressive capabilities across various domains.
- ▶ It’s completely closed-source: we do not know its architecture, training data composition, number of parameters, etc.
- ▶ Other models, such as Google’s Gemini, Mistral Large, Anthropic’s Claude, have approached its results in technical evaluations, but so far GPT-4 is the clear winner.
- ▶ It still has its problems, however: struggling with logical reasoning, copying its input to output, and most importantly, it generates false information.

Summary

- ▶ There you have it: a brief summary of modern neural architectures for NLP (and sequential data more broadly).
- ▶ Some interesting tasks they support:
 - ▶ Classification-based tasks: classification of individual tokens (Named Entity Recognition), or text fragments (Sentiment Analysis). These are tasks from the times of classical NLP.
 - ▶ Summarization: generate a summary of the input document. This task was attempted since the 1960s (by extracting salient sentences from text), and finally solved with high enough accuracy only now.
 - ▶ Question answering: given a document and a question, generate an answer.
 - ▶ Machine translation
 - ▶ Code generation: Copilot
 - ▶ Music generation: <https://openai.com/research/jukebox>